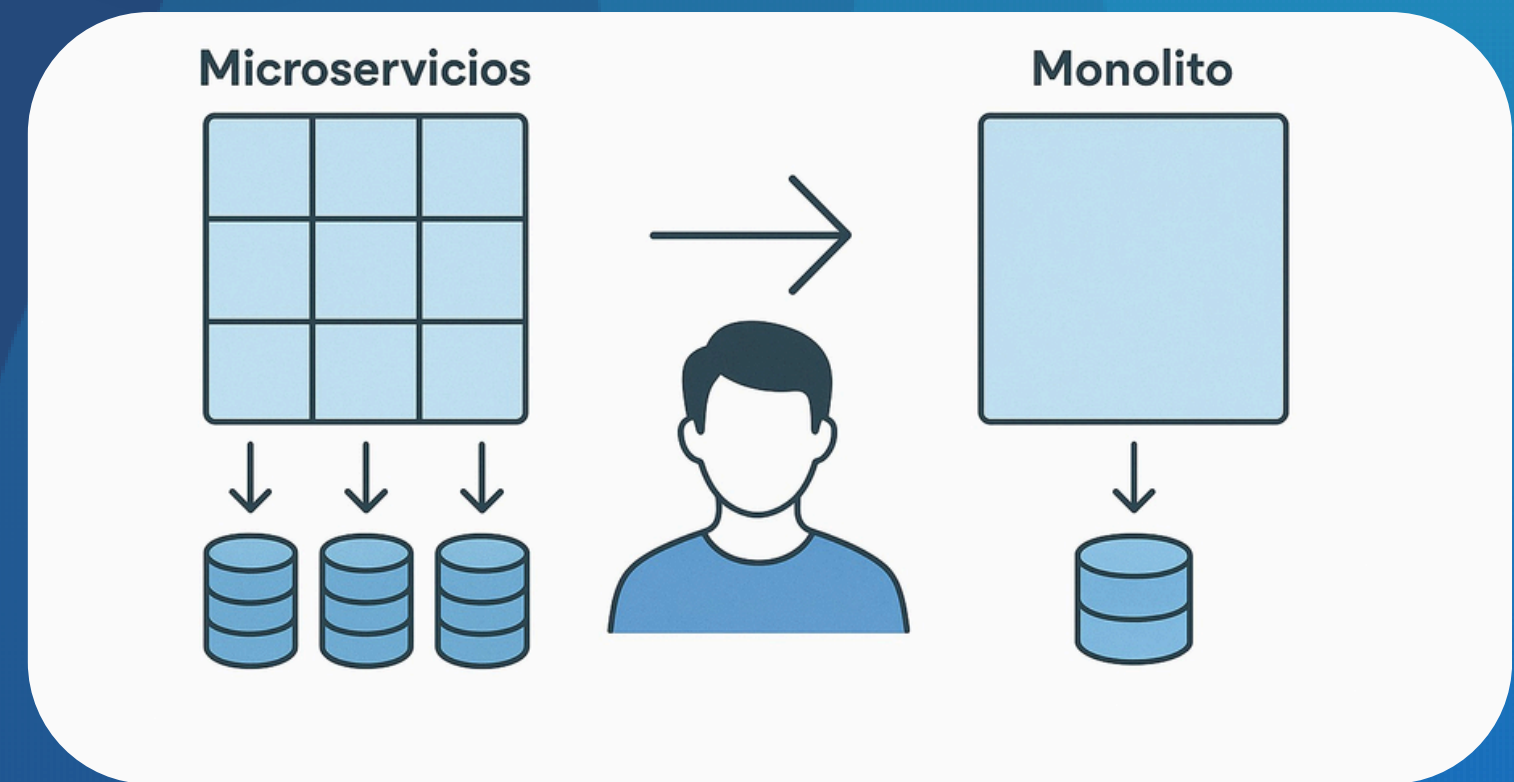


MONOLITO

Estilo monolítico: Sistema de Gestión de Pedidos sobre ASP.NET Core MVC, C# y SQL Server 2022.



CONTENIDO

01

**El estilo
monolítico**

03

**Análisis
arquitectónico**

05

Lecciones

02

**El stack
tecnológico**

04

**Diseño y
Resultados**

06

Conclusiones

EL ESTILO MONOLÍTICO

DEFINICIÓN

Toda la aplicación se construye, despliega y ejecuta como **una sola unidad**. Sus capas se separan en el código, pero viven juntas en el mismo proceso.

01

Un solo artefacto

Se compila y publica todo junto

02

Llamadas en memoria

Las capas se comunican sin red

03

Una base de datos compartida

Todos los módulos usan el mismo esquema

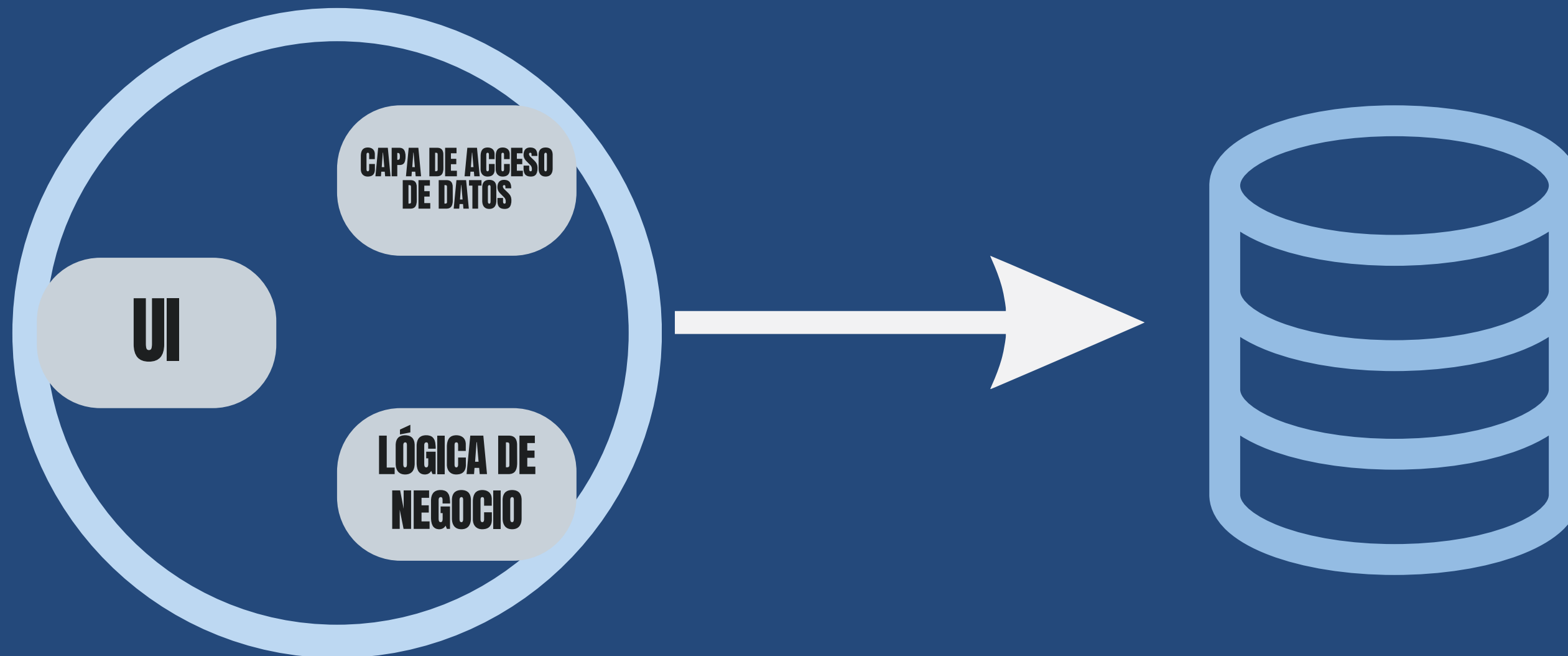
04

Escala como un todo

Para crecer se replica la aplicación completa

CARACTERÍSTICAS PRINCIPALES

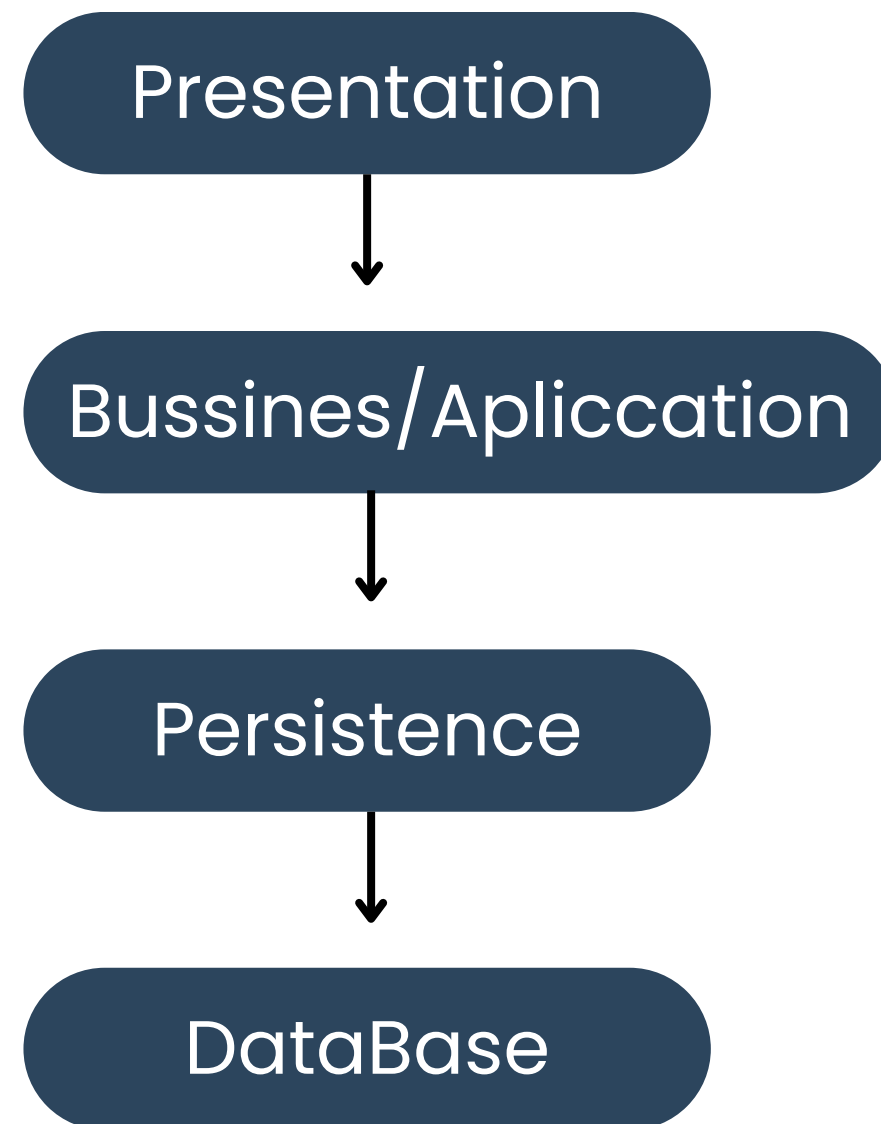
Un único proceso de ejecución y un único artefacto de despliegue.
Comunicación interna por llamada a función, no por red.



Una sola base de datos compartida por toda la aplicación.
El escalado replica la aplicación completa, no módulos sueltos.

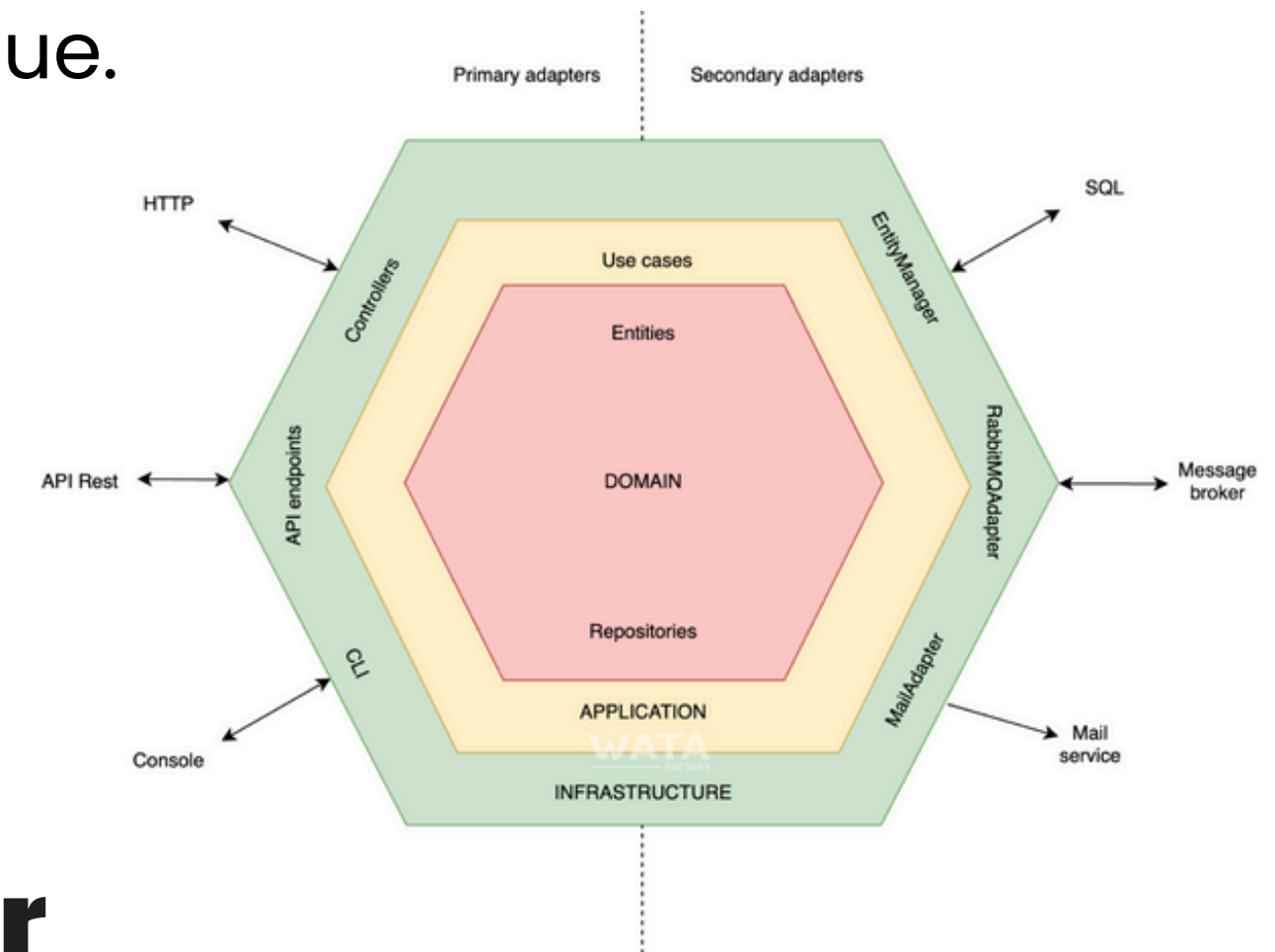
Layered

Separación horizontal por responsabilidad



Hexagonal / Clean

Separación por dependencia hacia el dominio, dentro de un único despliegue.



Monolito modular

Módulos de negocio con fronteras claras que comparten proceso y base de datos.

CLASIFICACIÓN DEL ESTILO

Historia y evolución

70-80 2009 2014 2023 2026

Monolito

Tres capas

Cliente-servidor:
presentación, lógica y datos
en una sola aplicación.

Monolito

ASP.NET MVC

Toda la aplicación web en un
solo proyecto.

Distribuido

Microservicios

Se divide el sistema para
escalar equipos por
separado.

Monolito

Prime Video

Vuelta al monolito para
reducir costos y
complejidad.

Hoy

Monolito modular

Un solo despliegue, con
módulos de límites claros.

Frameworks como Java EE, Rails, Django y ASP.NET MVC nacieron con esta filosofía. Desde 2020, el monolito bien diseñado se reevalúa como una opción legítima.

VENTAJAS Y DESVENTAJAS

A favor

- Un único repositorio y artefacto.
- Transacciones ACID directas, sin Saga.
- Latencia mínima entre capas.
- Despliegue y monitoreo simples.
- Curva de aprendizaje corta.



En contra

- No se escala un módulo por separado.
- Cada cambio redespliega todo.
- Un mismo lenguaje para toda la aplicación.
- Sin disciplina, degrada.
- Un fallo puede tumbar el proceso completo.

Cuándo usarlo

Equipos pequeños

Dos a diez desarrolladores que necesitan iterar rápido.



MVP y consistencia fuerte

Dominio aún cambiante y transacciones atómicas entre entidades.

Cuándo no Usar

Cargas muy distintas

Módulos que deben escalarse de forma independiente.

Muchos equipos autónomos

Despliegues frecuentes sin coordinación y aislamiento estricto de fallos.

Casos de la Industria

EMPRESA	STACK	QUÉ DEMUESTRA
Stack Overflow	Monolito .NET	Millones de solicitudes diarias con muy pocos servidores.
Shopify	Rails modular	Monolito modular deliberado a escala masiva.
Basecamp	Rails	Monolito por decisión filosófica, documentada por sus fundadores.
Amazon Prime Video	Monitoreo de video	Volvió de microservicios a monolito en 2023.

90%

de reducción de costos de infraestructura reportó Amazon Prime Video al migrar su servicio de monitoreo de audio y video desde microservicios hacia un monolito (2023).

EL STACK TECNOLÓGICO

¿Por qué este stack y no otro?

01

ASP.NET Core MVC, no Node/Spring

Frontend y backend en un solo proyecto, gratis y open source; en benchmarks TechEmpower está entre los frameworks web más rápidos, superando a Node y Spring en varios escenarios.

02

C#, no Java

Tipado estático fuerte e integración nativa con ASP.NET y EF Core, al ser del mismo fabricante; TIOBE lo nombró Lenguaje del Año 2025 por su crecimiento.

03

SQL Server 2022, no PostgreSQL/MySQL

ACID completo e integración de fábrica con EF Core; es la combinación canónica del ecosistema Microsoft, la que ya usan banca y seguros en LATAM.

Integración por llamado a función

Integración por llamado a función



Costo del stack: hoy y en producción

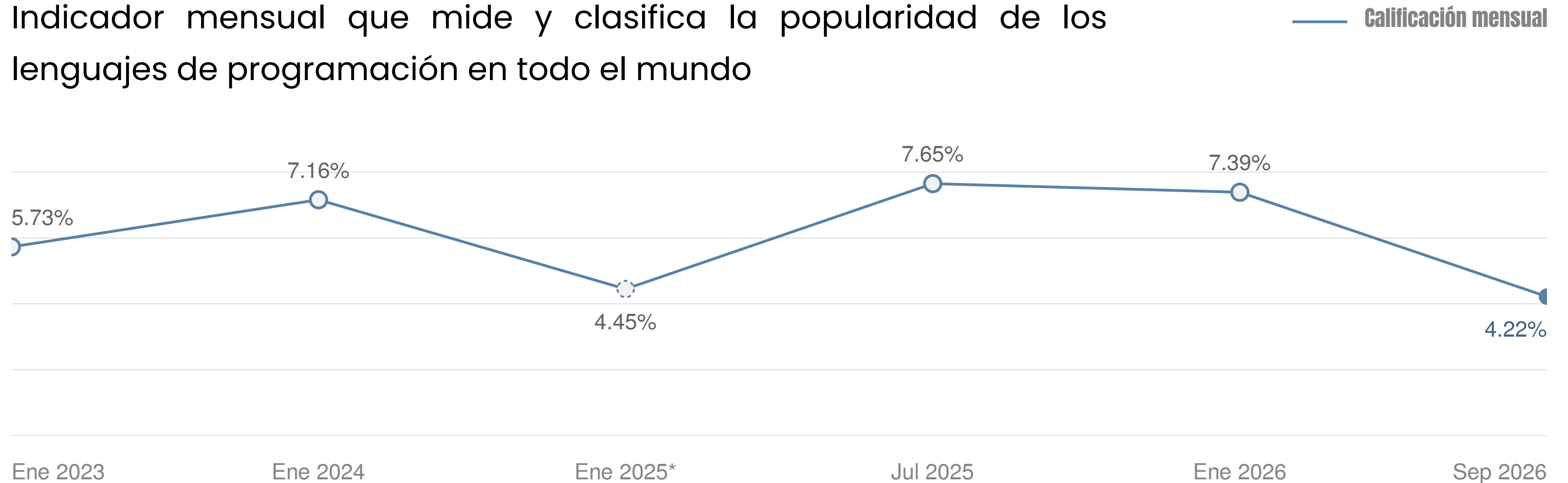
TECNOLOGÍA	PROYECTO	EN PRODUCCIÓN
.NET SDK, ASP.NET Core y C#	Gratis, open source (MIT)	Gratis siempre, en cualquier escala
Visual Studio	Community, gratis (uso académico)	Professional \$45/mes · Enterprise \$250/mes por usuario
SQL Server 2022	Express, gratis (límite ~10 GB)	Standard ~\$3,945 · Enterprise ~\$15,123 por 2 núcleos
Docker Compose	Desktop/Engine, gratis (uso personal)	Business ~\$24/usuario/mes en empresas de +250 empleados

¿Cuánto gana un desarrollador de este stack?

TECNOLOGÍA	PROYECTO	EN PRODUCCIÓN
.NET — Junior	USD 800–960 (~COP 3.2M–3.8M)	Talently, 2026
.NET — Semi-senior	USD 1,000–1,200 (~COP 4M–4.8M)	Talently, 2026
.NET — Senior	USD 1,300–1,560+ (~COP 5.2M–6.2M)	Talently, 2026
Promedio general (todo perfil)	~COP 3,018,247–3,130,021	Indeed Colombia, 2026

Popularidad de C# en el índice TIOBE

Indicador mensual que mide y clasifica la popularidad de los lenguajes de programación en todo el mundo



C# se mantiene 5.º global; Lenguaje del Año 2023 y 2025. *Ene 2025 estimado.

Mercado laboral

29,9%

de desarrolladores profesionales usó C# de forma extensiva (Stack Overflow 2025).

+22,2%

nuevos contribuidores a proyectos C# en GitHub, 2024–2025 (Octoverse).

52.000

ofertas de empleo mencionaron C# en 2025 (DevJobsScanner).

ANÁLISIS ARQUITECTÓNICO

Atributos de calidad ISO/IEC 25010

Favorece

- Adecuación funcional
- Eficiencia de desempeño
- Capacidad de interacción
- Mantenibilidad.

Depende

- Compatibilidad
- Fiabilidad
- Seguridad
- Protección (safety).

Limita

- Flexibilidad: no se puede escalar ni redespregar solo el módulo de Pedidos.

Principios de diseño

PRINCIPIO	CUMPLIMIENTO	EVIDENCIA EN EL PROYECTO
SOLID · SRP	✓ Alto	Un Service por responsabilidad de negocio
SOLID · DIP	✓ Alto	Los Controllers dependen de interfaces inyectadas en Program.cs
DRY	≈ Medio	Validaciones de existencia repetidas en varios Services
YAGNI	✓ Alto	Sin mensajería, colas ni caché distribuido
OCP	≈ Medio	nuevos estados exigen tocar el enum

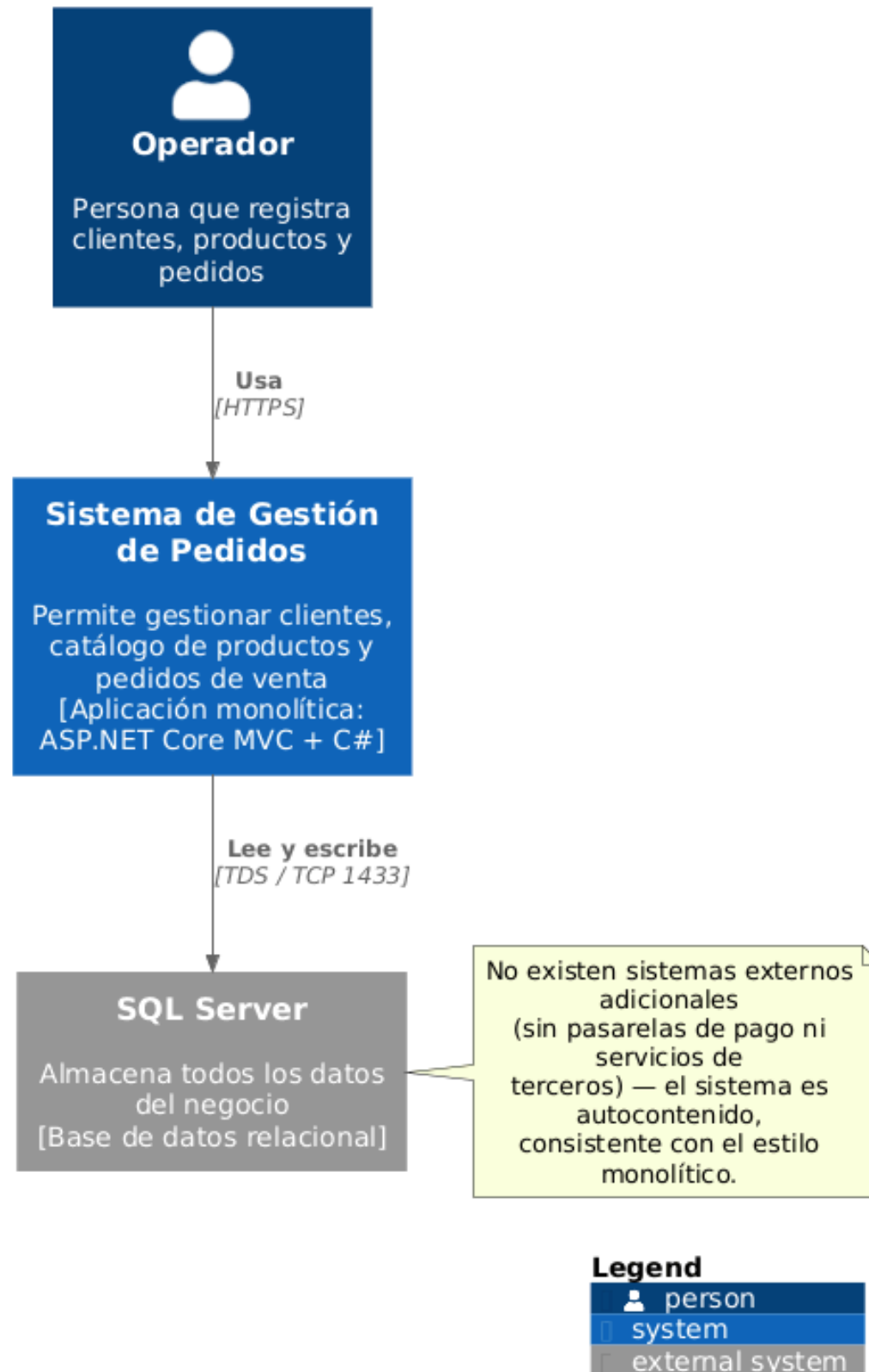
Decisiones de arquitectura

DECISIÓN	CONTEXTO	MECANISMO	CONSECUENCIA
Monolito en capas	Equipo de 3, plazo corto	Controllers, Services, Data	Simplicidad sobre escalado
Transacciones explícitas	Stock y total atómicos	Begin, Commit, Rollback	Sin estados inconsistentes
Excepciones de dominio	Errores claros al usuario	3 excepciones tipadas	Captura en cada Controller
Docker Compose	Contenedores obligatorios	app y db separados	Un proceso por contenedor

DISEÑO Y RESULTADOS

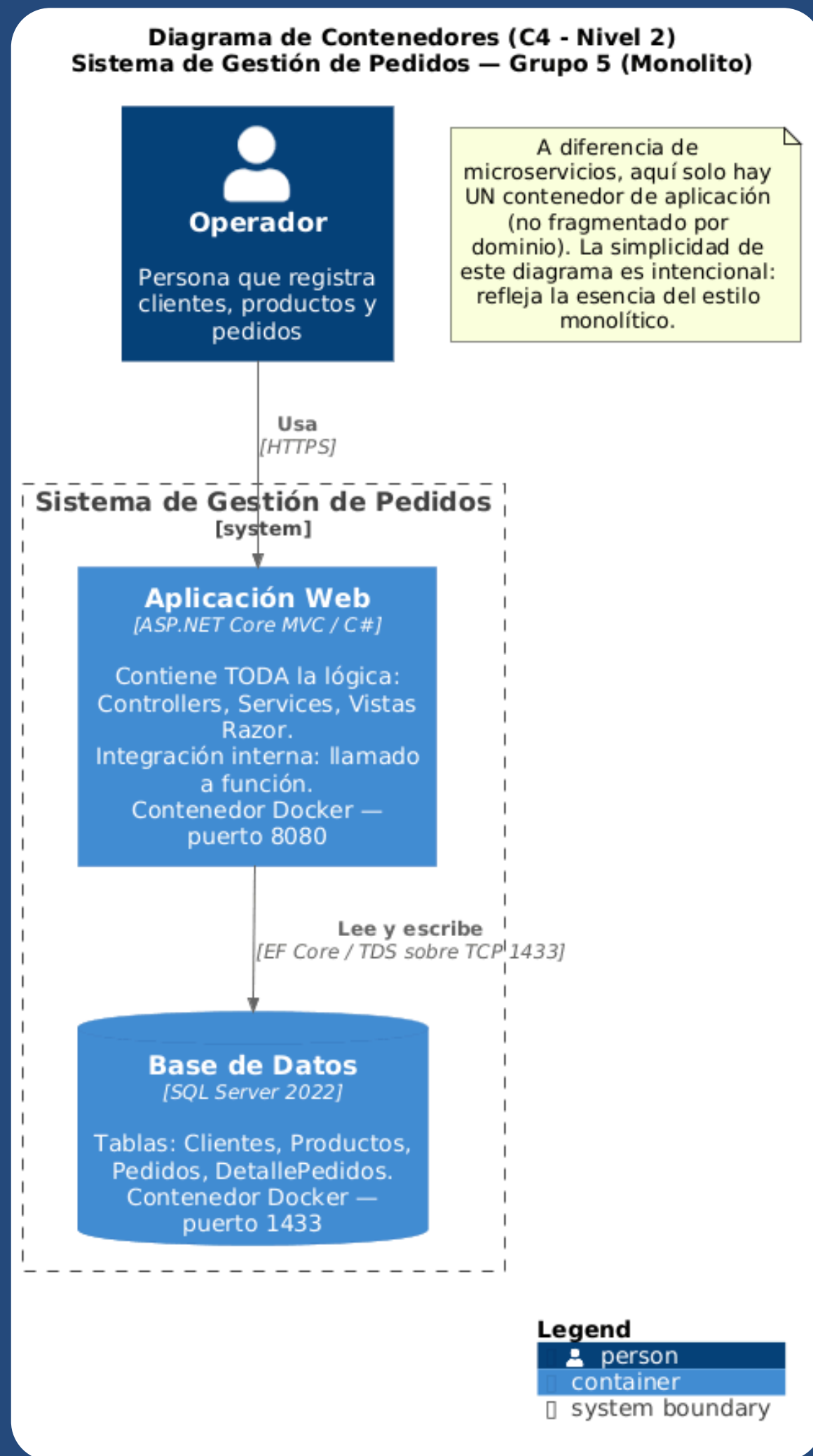
Diagrama de contexto

Diagrama de Contexto (C4 - Nivel 1)
Sistema de Gestión de Pedidos — Grupo 5 (Monolito)



Un solo contenedor de aplicación con toda la lógica. Solo la base de datos, con estado, vive aparte.

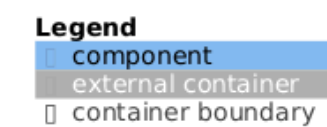
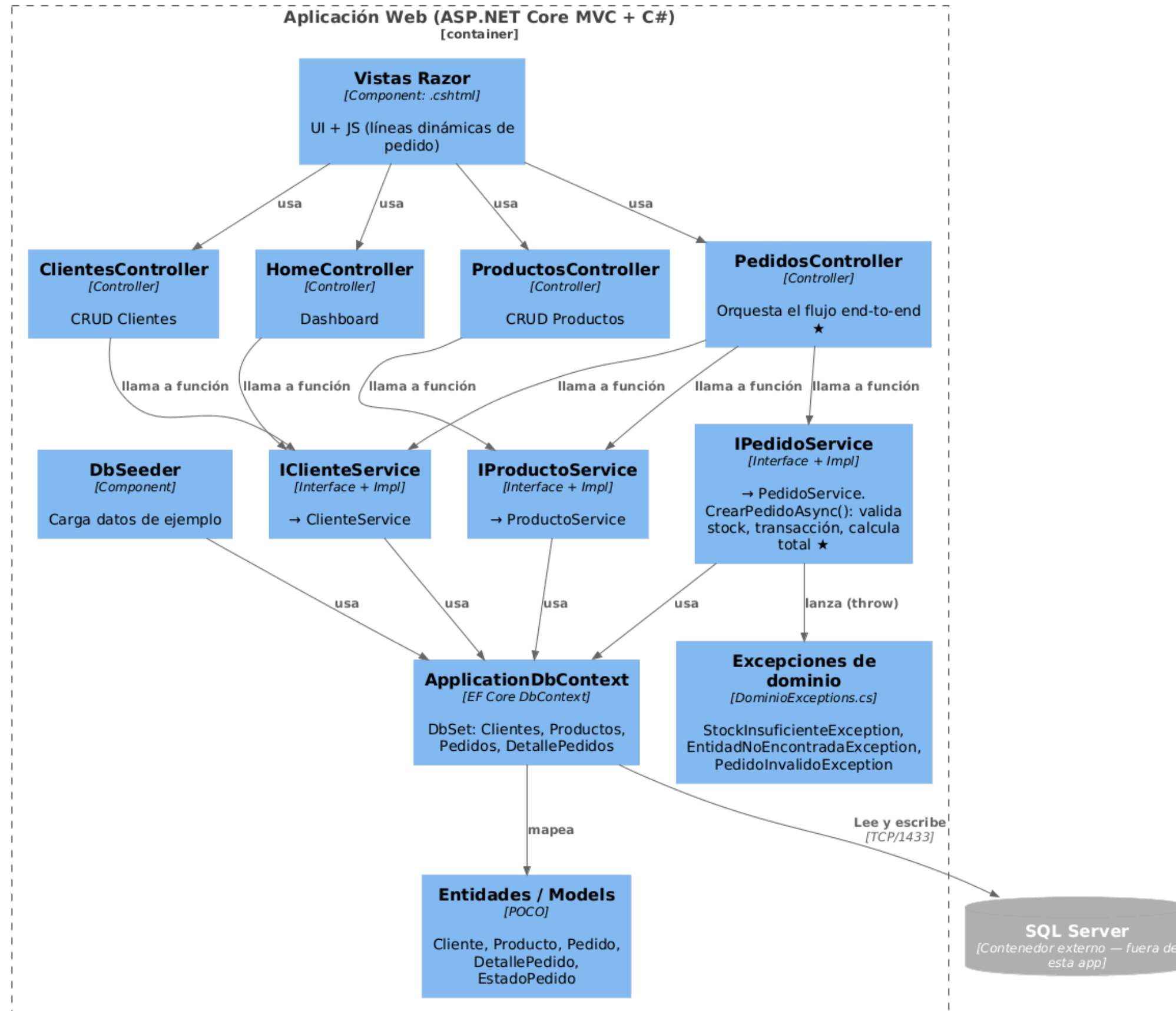
Diagrama de Contenedores



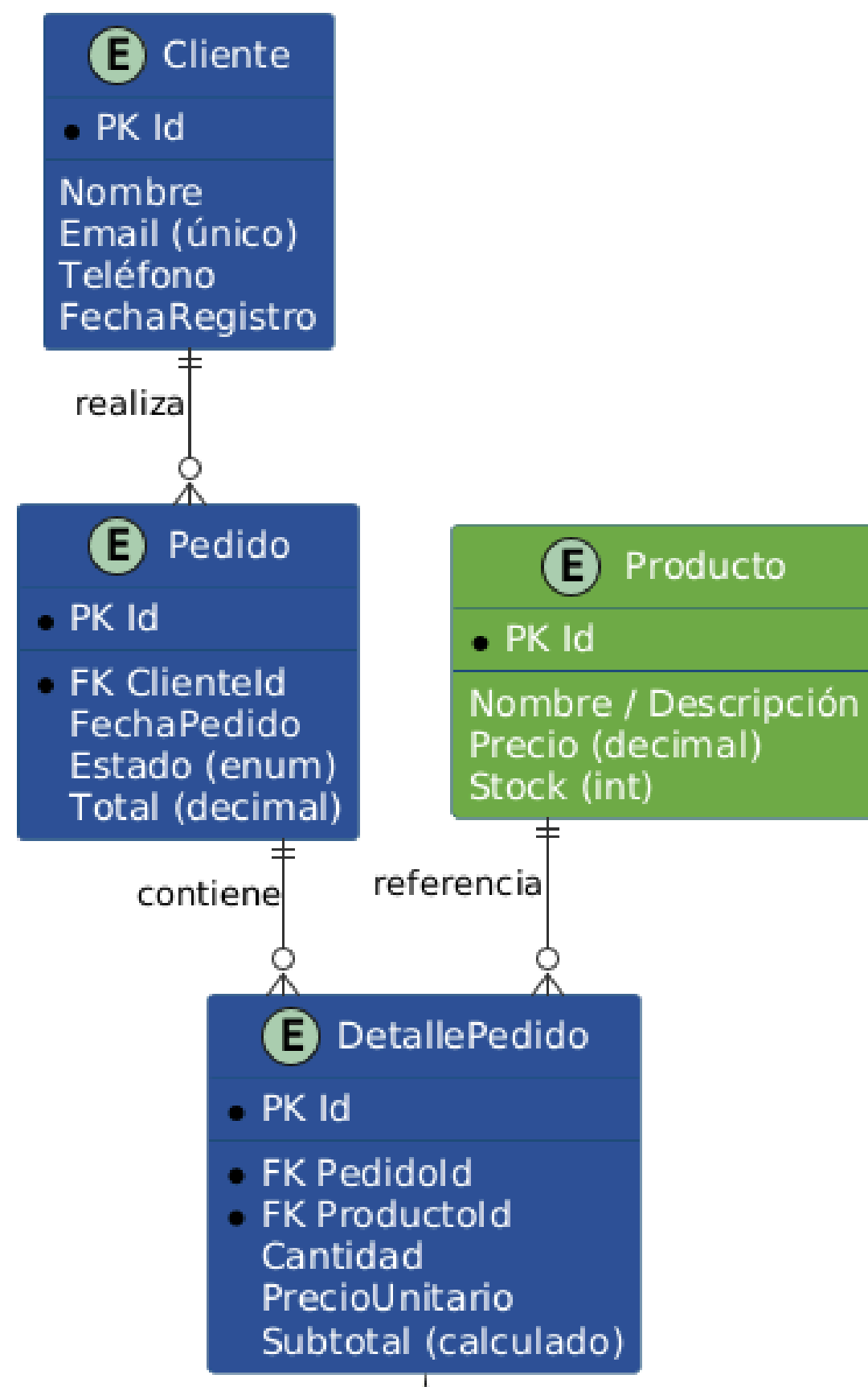
Un solo contenedor de aplicación con toda la lógica. Solo la base de datos, con estado, vive aparte.

Diagrama de Componentes

Diagrama de Componentes (C4 - Nivel 3, opcional)
Contenedor: Aplicación Web (ASP.NET Core MVC) — Grupo 5



Modelo de datos



Cuatro entidades interrelacionadas, una más que el mínimo exigido por el entregable.

LECCIONES APRENDIDAS

Cinco aprendizajes del ejercicio, más allá del documento entregado.

Qué se implementó

Flujo end-to-end

Del formulario a la base de datos, con descuento de inventario verificable.

Persistencia real

EF Core 8, migraciones code-first y un DbSeeder al iniciar el contenedor.

Robustez

Data Annotations, reglas de negocio en los Services y rollback ante cualquier fallo.

Lecciones aprendidas

Sin versionado de APIs ni descubrimiento de servicios, el caso de uso avanzó más rápido.

Las **transacciones explícitas** fueron más fáciles de depurar que la consistencia eventual.

El healthcheck de SQL Server en Compose fue el punto más delicado del arranque.

Documentar ADRs dio claridad compartida y preparó la sustentación.

“Monolito y buen diseño no son mutuamente excluyentes.”



Explorer

...

> SistemaG Explorer (Ctrl+Shift+E)

> Outline

> Timeline



Open Chat

Ctrl + Alt + I

Show All Commands

Ctrl + Shift + P

Open Settings

Ctrl + ,

REFERENCIAS

01

Richards, M., & Ford, N. (2020). Fundamentals of software architecture: An engineering approach. O'Reilly Media.

02

Newman, S. (2019). Monolith to microservices: Evolutionary patterns to transform your monolith. O'Reilly Media.

03

Fowler, M. (2015, 3 de junio). MonolithFirst. martinowler.com. Pasar de "el sistema debe ser confiable" a "RTO de 5 minutos y RPO de 30 segundos" fue lo que hizo el diseño discutible y evaluable.

04

Kolny, M. (2023, 22 de marzo). Scaling up the Prime Video audio/video monitoring service and reducing costs by 90%. Prime Video Tech. Escribir las siete tensiones nos obligó a admitir qué estábamos cediendo en cada decisión, en lugar de asumir que todo se cumplía a la vez.

05

Westeinde, K. (2019, 21 de febrero). Deconstructing the monolith: Designing software that maximizes developer productivity. Shopify Engineering. Las decisiones de safety exigen determinismo y explicabilidad, y eso terminó siendo un componente separado con su propio ciclo de validación.

06

TIOBE Software. (2026, enero). TIOBE Index for January 2026. [Richards, M., & Ford, N. \(2020\). Fundamentals of software architecture: An engineering approach. O'Reilly Media.](#)

07

Microsoft. (s. f.). SQL Server 2022 pricing. [Newman, S. \(2019\). Monolith to microservices: Evolutionary patterns to transform your monolith. O'Reilly Media.](#)

08

International Organization for Standardization. (2023). ISO/IEC 25010:2023 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model. [Fowler, M. \(2015, 3 de junio\). MonolithFirst. martinowler.com. Pasar de "el sistema debe ser confiable" a "RTO de 5 minutos y RPO de 30 segundos" fue lo que hizo el diseño discutible y evaluable.](#)

09

Bass, L., Clements, P., & Kazman, R. (2021). Software architecture in practice (4.ª ed.). Addison-Wesley.

10

Kreps, J. (2014, 2 de julio). Questioning the Lambda Architecture. O'Reilly Radar. [Westeinde, K. \(2019, 21 de febrero\). Deconstructing the monolith: Designing software that maximizes developer productivity. Shopify Engineering. Las decisiones de safety exigen determinismo y explicabilidad, y eso terminó siendo un componente separado con su propio ciclo de validación.](#)

GRACIAS